Primeiro Código: Guia Prático de Lógica e Python

Sua porta de entrada para o desenvolvimento de software

Aprenda a pensar como um programador e construa suas primeiras aplicações do zero.

Primeiro Código: Guia Prático de Lógica e Python
Autor: Arthur Magno Cano Garcia
Ano de publicação: 2026
Nota técnica: Os exemplos e projetos apresentados neste e-book foram desenvolvidos e testados utilizando Python 3 e o ambiente de desenvolvimento Visual Studio Code (VS Code).

Primeiro Código

Guia Prático de Lógica e Python

Sumário

Módulo / Capítulo	Tópicos de Aprendizado
Capítulo 1: Apresentação e Introdução	O que é programação, por que escolher Python e como configurar o seu ambiente de desenvolvimento sem complicações.
Capítulo 2: Conceitos Fundamentais	O que é lógica de programação, o conceito de algoritmos e como funcionam as variáveis e os tipos de dados.
Capítulo 3: Primeiros Passos com Python	Escrevendo as primeiras linhas de código, trabalhando com exibição de mensagens e entrada de dados do usuário.
Capítulo 4: Operadores e Expressões	Manipulação de operações matemáticas, comparações lógicas e tomada de decisões através de expressões.

Módulo / Capítulo	Tópicos de Aprendizado
Capítulo 5: Estruturas de Controle	Direcionando o fluxo do programa com estruturas condicionais (`if/else`) e laços de repetição (`while` e `for`).
Capítulo 6: Estruturas de Dados Básicas	Como armazenar e manipular coleções de dados utilizando Listas, Tuplas e Dicionários.
Capítulo 7: Organizando o Código	Criação de funções, passagem de parâmetros, retorno de valores e boas práticas de modularização.
Capítulo 8: Tratamento de Erros e Boas Práticas	Identificação e prevenção de falhas no código com `try/except`, depuração e padrões de código limpo.
Capítulo 9: Projeto Prático Final	Desenvolvimento completo de uma aplicação real passo a passo, consolidando todo o conteúdo estudado.

Bem-vindo à sua jornada na programação

Se você decidiu abrir este livro, parabéns: este é o primeiro passo para aprender a linguagem que move boa parte da tecnologia atual. É normal sentir alguma incerteza no começo. Telas pretas cheias de texto, termos em inglês e o mito de que programar exige ser um gênio da matemática costumam afastar muita gente. Mas a realidade é bem mais simples: programar é aprender a organizar seus pensamentos e dar instruções claras para o computador. Se você consegue estruturar ideias para resolver problemas do dia a dia, já tem o necessário para começar.

Este guia foi pensado para ajudar quem está dando os primeiros passos na área de Tecnologia da Informação. Em vez de teorias longas e termos complicados, o foco aqui é construir uma base prática em lógica de programação usando Python, uma

linguagem moderna, muito usada no mercado e amigável para iniciantes. Cada conceito vem acompanhado de explicações diretas e exemplos do dia a dia.

Para aproveitar ao máximo o material, pratique desde o início. Ninguém aprende a programar só lendo, assim como ninguém aprende a dirigir apenas olhando o painel do carro. A cada novo tópico, abra seu editor de código, digite os exemplos, mude alguns valores e veja o que acontece. Testar, errar e corrigir faz parte do aprendizado. Lembre-se de que todo desenvolvedor experiente começou exatamente assim: tentando entender seu primeiro código. Mantenha a curiosidade, tenha paciência com seu ritmo e aproveite os estudos. Boas-vindas ao mundo da programação!
Lembre-se de que todo desenvolvedor experiente, por mais avançado que seja hoje, começou exatamente no mesmo ponto em que você está agora: tentando entender o seu primeiro algoritmo. Mantenha a curiosidade acesa, seja paciente com o seu processo e aproveite a jornada. Seja muito bem-vindo ao mundo do desenvolvimento de software.

Capítulo 1: Apresentação e Introdução

Quando se fala em programação, muita gente pensa naquela cena clássica de filme: uma sala escura, várias telas com linhas verdes subindo rápido e alguém digitando sem parar para invadir um sistema em poucos segundos. Essa imagem funciona bem no cinema, mas o dia a dia de quem programa é bem diferente e muito mais prático. Programar não tem a ver com decifrar códigos secretos ou ter um dom para matemática. No fundo, é sobre resolver problemas e comunicar ideias. Trata-se de pegar um desafio, dividi-lo em etapas menores e passar instruções claras e ordenadas para o computador. A máquina executa tudo com muita rapidez, mas depende inteiramente das ordens que recebe.
Programar não é sobre decifrar códigos secretos ou ter uma habilidade matemática genial fora do comum. Em essência, programar nada mais é do que resolver problemas e se comunicar. É a arte de pegar um desafio do mundo real, dividi-lo em partes menores e fornecer um conjunto de instruções claras, lógicas e ordenadas para que um computador possa executá-las. A máquina é incrivelmente rápida e precisa, mas não possui iniciativa própria, ela depende inteiramente da clareza das ordens que recebe.

O papel da tecnologia no seu dia a dia

Mesmo que você nunca tenha escrito uma linha de código, você já usa software o tempo todo. Ao abrir um aplicativo de filmes ou músicas e ver recomendações, há um programa analisando suas preferências. Quando você pede comida por um aplicativo,

sistemas calculam o trajeto, conectam você ao restaurante e acompanham a entrega. Jogos online conectam pessoas do mundo todo em tempo real, e ferramentas de busca respondem a perguntas em segundos.
Toda essa estrutura foi criada por pessoas que um dia tomaram a mesma decisão que você: aprender a programar. Entender como a tecnologia funciona permite que você deixe de ser apenas um usuário e passe a criar suas próprias soluções, seja para facilitar sua rotina ou para trabalhar no mercado de tecnologia.
Toda essa infraestrutura invisível que sustenta a sociedade moderna foi construída por pessoas que um dia tomaram a mesma decisão que você está tomando agora: aprender a programar. Entender a tecnologia permite que você deixe de ser apenas um consumidor passivo das ferramentas digitais e se torne um criador, capaz de desenvolver soluções reais para problemas da sua vida, da sua comunidade ou de grandes empresas.

Por que aprender Python?

Existem muitas linguagens de programação, mas poucas são tão indicadas para quem está começando quanto Python. Ela foi criada para ser fácil de ler e escrever, com uma estrutura muito próxima do inglês simples. Isso ajuda você a gastar menos tempo decorando regras difíceis e focar no que realmente importa: a lógica para resolver cada problema.
Embora seja simples de aprender, Python é uma linguagem completa e usada por grandes empresas em diversas áreas:
Apesar de ser simples de aprender, Python está longe de ser uma ferramenta limitada ou meramente acadêmica. Ela é uma das linguagens mais utilizadas por gigantes da tecnologia e está presente nos setores mais inovadores do mercado de trabalho atual:

- **Automação de tarefas:** Criação de scripts para realizar trabalhos repetitivos, como organizar arquivos, enviar e-mails e atualizar planilhas.
- **Desenvolvimento Web:** Construção de sites, sistemas e APIs usando ferramentas como Django e Flask.
- **Ciência e Análise de Dados:** Organização de grandes volumes de informação e geração de relatórios para tomada de decisão.
- **Inteligência Artificial:** Uso amplamente difundido para criar e treinar modelos de aprendizado de máquina.

Como o aprendizado é rápido, você consegue ver os primeiros resultados logo nas primeiras semanas de estudo.

O próximo passo

Agora que vimos o papel de Python, é hora de ir para a prática. Antes de escrever o código em si, precisamos entender a base de qualquer programa: a lógica. No próximo capítulo, vamos ver como organizar o raciocínio em passos claros para resolver problemas de forma estruturada.

Capítulo 2: Conceitos Fundamentais

Com a introdução ao Python feita, o próximo passo é organizar a forma de pensar. Antes de lidar com pontuação ou comandos específicos, a parte principal do trabalho está em analisar o problema e planejar os passos da solução de um jeito que o computador consiga entender.

1. O que é Lógica e Pensamento Computacional

Pensamento computacional é a capacidade de formular problemas de modo que seja possível resolvê-los com o apoio de computadores. Essa forma de pensar costuma ser dividida em quatro partes principais:

- **Decomposição:** Dividir um problema complexo em etapas menores. Por exemplo, em um aplicativo de entregas, o processo envolve receber o pedido, confirmar o pagamento, avisar o restaurante e chamar o entregador.
- **Reconhecimento de Padrões:** Perceber situações parecidas que já foram resolvidas antes. Se a validação de pagamento é a mesma para vários tipos de compra, você reutiliza a mesma lógica.
- **Abstração:** Focar no que é importante para a solução e ignorar detalhes irrelevantes. Um mapa de navegação mostra as ruas e o trânsito, mas ignora a cor dos prédios ou das casas.
- **Algoritmos:** Montar uma sequência clara de passos para chegar ao resultado esperado.

2. Algoritmos do Dia a Dia

Nós usamos algoritmos todos os dias sem perceber. Uma receita de cozinha, o passo a passo para montar um móvel ou o caminho até o trabalho são exemplos disso. Veja como ficaria a sequência para ligar um alarme residencial:

- 1. Verificar se todas as portas e janelas estão fechadas.

- 2. Se houver alguma aberta, emitir aviso e aguardar o fechamento.
- 3. Solicitar a senha de quatro dígitos no painel.
- 4. Se a senha estiver correta, iniciar contagem regressiva de 30 segundos.
- 5. Ativar os sensores de movimento.

A importância da clareza nas instruções
As pessoas entendem contextos com facilidade. Se você pede a alguém para "buscar pão", a pessoa sabe que deve pegar o dinheiro, ir à padaria, escolher o pão, pagar e voltar. Para um computador, porém, essa instrução é genérica demais.

Se uma ordem for confusa ou faltar alguma etapa, a máquina executará exatamente o que foi escrito, gerando erros (bugs). Se esquecermos de mandar o sistema verificar o saldo antes de aprovar uma compra, ele pode liberar o pedido sem receber o pagamento. O computador não adivinha intenções; ele só segue ordens.

3. Introdução a Fluxogramas

Um fluxograma é um desenho do algoritmo. Ele usa formas geométricas simples para mostrar o caminho das informações em um programa, indicando onde o processo começa, por onde passa e onde termina.

As formas básicas de um fluxograma são:

- **Terminal (Elipse / Retângulo Arredondado):** Indica o ponto de Início ou de Fim do processo.
- **Entrada / Saída de Dados (Paralelogramo):** Representa o momento em que o sistema recebe uma informação (como uma senha digitada) ou exibe um resultado na tela.
- **Processamento / Ação (Retângulo):** Indica uma operação realizada pelo sistema, como um cálculo matemático ou a atualização de um dado.
- **Decisão / Condição (Losango):** Representa um ponto em que o fluxo toma caminhos diferentes com base em uma pergunta que pode ser respondida com "Sim" ou "Não" (Verdadeiro ou Falso).

Exemplos Estruturados para Mapeamento:

Exemplo 1: Processo de Login no Sistema
[Início: Usuário abre o app]
→ [Entrada: Digitou E-mail e Senha]
→ [Decisão: A senha corresponde ao cadastro?]
|—→ (Sim) → [Processo: Carregar Perfil] → [Saída: Exibir Tela Principal] → [Fim]

    |—> (Não) → [Saída: Exibir "Senha Incorreta"] → [Início: Solicitar Dados Novamente]

Exemplo 2: Verificação de Saldo para Saque Bancário
[Início: Inseriu o cartão no caixa eletrônico]
→ [Entrada: Digitou o valor desejado para saque]
→ [Decisão: O saldo da conta é maior ou igual ao valor do saque?]
    |—> (Sim) → [Processo: Deduzir valor do saldo] → [Saída: Entregar cédulas ao cliente] → [Fim]
    |—> (Não) → [Saída: Exibir mensagem "Saldo Insuficiente"] → [Fim]

Capítulo 3: Primeiros Passos com Python

Com os conceitos iniciais em mente, é hora de começar a prática. Neste capítulo, você vai preparar o ambiente no computador, escrever seus primeiros comandos em Python, salvar informações na memória e interagir com o programa.

1. Configurando o Ambiente de Desenvolvimento

Para rodar programas em Python, precisamos de duas coisas: o interpretador da linguagem (que traduz o código para o computador) e um editor de código (onde você escreve os programas).

Passo 1: Instalando o Python

- Acesse o site oficial (python.org) e faça o download da versão estável mais recente para o seu sistema operacional.
- **Atenção no Windows:** Na primeira tela do instalador, marque a opção `"Add Python.exe to PATH"` antes de avançar. Isso permite rodar os comandos do Python direto pelo terminal.
- No macOS e Linux, o Python costuma vir instalado, mas é recomendável atualizar para a versão 3.x mais recente pelo site oficial.

Passo 2: Instalando e Configurando o VS Code
O Visual Studio Code (VS Code) é um dos editores mais usados para programar. Ele é gratuito, leve e tem vários recursos úteis para o dia a dia.

- Baixe e instale o VS Code acessando code.visualstudio.com.
- Após abrir o editor, clique no ícone de Extensions (ou pressione Ctrl+Shift+X) e pesquise por **"Python"** (desenvolvida pela Microsoft). Clique em "Install".

- Crie uma pasta no seu computador chamada `estudos_python`. No VS Code, vá em **File > Open Folder** e selecione essa pasta.
- Crie o seu primeiro arquivo clicando no ícone de "New File" e nomeie-o como `inicio.py`. A extensão `.py` indica que se trata de um arquivo de código Python.
- Para executar o código, você usará o terminal integrado do VS Code (abra usando o atalho Ctrl + ` ou acessando Terminal > New Terminal).

2. Sintaxe Básica e Comandos de Entrada e Saída

Em Python, a interação básica acontece mostrando mensagens na tela e recebendo dados digitados pelo usuário.

Exibindo dados com print()
O comando `print()` serve para exibir textos ou resultados. Tradicionalmente, o primeiro exemplo em qualquer linguagem mostra uma mensagem simples de boas-vindas na tela.

```python
print("Olá, Mundo!")
```

Ao executar essa linha no VS Code (digitando `python inicio.py` no terminal), a expressão entre aspas será impressa na tela.

Recebendo dados com input()
Para pedir informações ao usuário, usamos o comando `input()`. Ele faz o programa pausar até que a pessoa digite algo e aperte Enter.

```python
input("Digite o seu nome: ")
```

3. Variáveis e Tipos de Dados

Uma variável é um espaço na memória para guardar informações que o programa precisa usar depois. Pense em uma caixa com etiqueta: o nome da etiqueta é o identificador e o que está dentro é o valor armazenado.

Tipos Primitivos de Dados

Os dados em Python são divididos em alguns tipos principais:

- **Inteiros (int):** Números inteiros, positivos ou negativos, sem casa decimal (ex: 25, -10, 0).
- **Decimais / Ponto Flutuante (float):** Números reais que possuem casas decimais, separados por ponto em vez de vírgula (ex: 1.75, 59.90, -3.14).
- **Textos / Strings (str):** Cadeias de caracteres delimitadas por aspas simples ou duplas (ex: "Maria", 'Python 3', "123").
- **Booleanos (bool):** Valores lógicos representados estritamente por `True` (Verdadeiro) ou `False` (Falso).

Python
```
# Exemplos de declaração de variáveis
idade = 28           # int
altura = 1.75        # float
nome = "Carlos"      # str
ativo = True         # bool
```

Conversão de Tipos (Type Casting)
Tudo o que vem do comando `input()` entra como texto (string). Se você quiser fazer contas com esse valor, precisa convertê-lo usando `int()` para números inteiros ou `float()` para números com decimal.

Python
```
# Convertendo a entrada para número inteiro
idade = int(input("Digite sua idade: "))

# Convertendo a entrada para número decimal
preco = float(input("Digite o valor do produto: "))
```

4. Formatando Textos com f-strings

A forma mais simples de misturar texto e variáveis em Python é usando **f-strings**. Basta colocar a letra `f` antes das aspas e inserir o nome da variável entre chaves `{}`.

```python
nome = "Ana"
idade = 22
cidade = "São Paulo"

# Exibindo variáveis formatadas dentro do texto
print(f"Olá, {nome}! Você tem {idade} anos e mora em {cidade}.")
```

Essa forma de escrita deixa o código mais fácil de ler e evita erros comuns ao juntar texto com números.

Mão no Código

Abra o VS Code, crie os arquivos indicados e pratique a escrita dos seus primeiros scripts:

- **Exercício 1 (saudacao.py):** Crie um script que peça o nome e a profissão da pessoa. Em seguida, mostre uma mensagem no terminal usando f-string (exemplo: "Seja bem-vindo, Lucas! É muito bom ter um Desenvolvedor por aqui.").
- **Exercício 2 (calculadora_ano.py):** Faça um programa que peça o ano de nascimento do usuário e o ano atual. Converta os valores para número inteiro, calcule a idade e exiba o resultado no terminal.

Capítulo 4: Operadores e Expressões

Até aqui, você viu como guardar dados em variáveis e mostrar mensagens. Agora é hora de fazer operações com esses dados. Os operadores servem para fazer cálculos, comparar valores e criar regras de decisão no código.

1. Operadores Aritméticos

Os operadores aritméticos fazem as contas matemáticas no programa. Além das operações básicas, o Python tem algumas opções úteis para o dia a dia da programação.

- **Soma (+):** Soma dois valores numéricos.

- **Subtração (-):** Subtrai o valor da direita do valor da esquerda.
- **Multiplicação (*):** Multiplica dois números.
- **Divisão Comum (/):** Divide dois números e sempre resulta em um valor decimal (float), mesmo que o resultado seja exato.
- **Divisão Inteira (//):** Divide dois números e descarta a parte decimal, retornando apenas o número inteiro resultante.
- **Resto da Divisão / Módulo (%):** Retorna o resto da divisão inteira entre dois números.
- **Potência (**):** Eleva um número à potência de outro.

```python
a = 10
b = 3

print(f"Soma: {a + b}")           # 13
print(f"Subtração: {a - b}")      # 7
print(f"Multiplicação: {a * b}")  # 30
print(f"Divisão: {a / b}")        # 3.3333...
print(f"Divisão Inteira: {a // b}")# 3
print(f"Resto da Divisão: {a % b}")# 1
print(f"Potência: {a ** b}")       # 1000
```

A utilidade prática do Módulo (%)
O operador de resto da divisão (%) ajuda muito na lógica do código. Um uso comum é saber se um número é par ou ímpar: qualquer número inteiro dividido por 2 que deixa resto 0 é par; se o resto for 1, é ímpar.

```python
numero = 15
resto = numero % 2
print(f"O resto da divisão de {numero} por 2 é: {resto}")
# Como o resto é 1, sabemos que 15 é ímpar.
```

Ordem de Precedência
O Python resolve as operações matemáticas seguindo esta ordem padrão:

- 1. Parênteses: ()
- 2. Potenciação: **

- 3. Multiplicação, Divisão, Divisão Inteira e Módulo: *, /, //, %
- 4. Adição e Subtração: +, -

Use parênteses quando quiser mudar a ordem natural das contas e garantir o resultado certo.

Python
```
# Sem parênteses: a multiplicação é feita primeiro (2 * 5 = 10 + 10 = 20)
resultado1 = 10 + 2 * 5

# Com parênteses: a soma é feita primeiro (10 + 2 = 12 * 5 = 60)
resultado2 = (10 + 2) * 5

print(f"Sem parênteses: {resultado1}")
print(f"Com parênteses: {resultado2}")
```

2. Operadores Relacionais (Comparação)

Os operadores de comparação servem para checar a relação entre dois valores. O resultado de uma comparação é sempre True (Verdadeiro) ou False (Falso).

- **Igual a (==):** Verifica se dois valores são iguais.
- **Diferente de (!=):** Verifica se dois valores são diferentes.
- **Maior que (>):** Verifica se o valor da esquerda é maior que o da direita.
- **Menor que (<):** Verifica se o valor da esquerda é menor que o da direita.
- **Maior ou igual a (>=):** Verifica se o valor da esquerda é maior ou igual ao da direita.
- **Menor ou igual a (<=):** Verifica se o valor da esquerda é menor ou igual ao da direita.

Atenção para não confundir: Atribuição (=) e Comparação (==)
Um erro comum no começo é confundir um sinal de igual com dois:

- **Um sinal de igual (=):** Guarda um valor na variável. Exemplo: x = 5 (armazena 5 em x).
- **Dois sinais de igual (==):** Checa se dois valores são iguais. Exemplo: x == 5 (pergunta se o valor em x é 5).

```python
idade = 18
meta_idade = 21

print(f"É maior de idade? {idade >= 18}")
print(f"Tem a idade exata da meta? {idade == meta_idade}")
print(f"A idade é diferente de 20? {idade != 20}")
```

3. Operadores Lógicos

Os operadores lógicos juntam mais de uma comparação na mesma regra. Eles são muito usados para validar condições em sistemas reais.

- **and (E):** Retorna True apenas se TODAS as condições forem verdadeiras.
- **or (OU):** Retorna True se pelo menos UMA das condições for verdadeira.
- **not (NÃO):** Inverte o resultado lógico (transforma True em False e vice-versa).

Exemplo Prático 1: Requisitos para Vaga de Estágio
Para concorrer a uma vaga de estágio, a pessoa precisa estar no ensino superior E saber Python.

```python
cursando_superior = True
sabe_python = True

elegivel_vaga = cursando_superior and sabe_python
print(f"Candidato elegível para a vaga? {elegivel_vaga}")
```

Exemplo Prático 2: Passar de Fase em um Jogo
Em um jogo, para avançar de fase o jogador precisa somar mais de 1000 pontos OU ter a chave secreta.

```python
pontuacao = 850
tem_chave_secreta = True
```

```
passou_fase = (pontuacao > 1000) or tem_chave_secreta
print(f"Jogador passou de fase? {passou_fase}")
```

Exemplo Prático 3: Inversão de Status com not

Python
```
sistema_bloqueado = False
acesso_permitido = not sistema_bloqueado

print(f"Acesso permitido ao sistema? {acesso_permitido}")
```

Mão no Código

Hora de praticar. Abra seu editor e crie o arquivo abaixo para testar os operadores.

- **Desafio Prático (analise_credito.py):** Crie um programa que simule a análise de um empréstimo bancário. O script deve receber a idade, a renda mensal e o valor da parcela desejada. A aprovação depende destas três condições simultâneas:
    - 1. Ter idade entre 18 e 70 anos (inclusive).
    - 2. Ter renda mensal de pelo menos 2500.00.
    - 3. A parcela não pode passar de 30% da renda mensal (parcela <= renda * 0.30).

Capítulo 5: Estruturas de Controle

Até agora, os scripts rodam de forma sequencial: linha por linha, do começo ao fim. No entanto, programas úteis precisam tomar decisões com base nos dados e repetir tarefas automaticamente. É aí que entram as estruturas de controle de fluxo.

1. Tomada de Decisão: if, elif e else

As condições permitem que o código escolha qual caminho seguir. Em Python, usamos as palavras if (se), elif (senão se) e else (senão) para construir essas regras.

Sintaxe, Indentação e Dois-pontos
Em vez de usar chaves para delimitar blocos, o Python usa dois-pontos (:) e recuo de texto (indentação, geralmente 4 espaços). A indentação mostra ao Python quais linhas pertencem a qual bloco.

```python
idade = int(input("Digite sua idade: "))

if idade >= 18:
    print("Você é maior de idade e tem acesso completo.")
elif idade >= 12:
    print("Você é adolescente e tem acesso limitado.")
else:
    print("Você é criança e seu acesso requer autorização.")
```

As condições são testadas na ordem em que aparecem. Assim que uma for verdadeira, o bloco correspondente roda e o resto é ignorado. Se nenhuma for atendida, o bloco else entra em ação.

2. Laços de Repetição Contada: for e range()

Quando você já sabe quantas vezes precisa repetir uma instrução, a escolha ideal é o laço for, que costuma ser usado junto com a função range().

Trabalhando com range()
A função range() aceita parâmetros de início, fim e passo. Vale notar que o número final fica de fora da contagem: range(1, 6) vai de 1 até 5.

```python
# Exemplo 1: Repetição simples de 0 a 4
for i in range(5):
    print(f"Execução número {i}")

# Exemplo 2: Tabuada do 5
numero = 5
for multiplicador in range(1, 11):
    resultado = numero * multiplicador
```

```
    print(f"{numero} x {multiplicador} = {resultado}")
```

3. Laços de Repetição Condicional: while, Loop Infinito e break

A estrutura while (enquanto) repete um bloco enquanto a condição continuar verdadeira. Ela é útil quando não sabemos de antemão quantas repetições vão acontecer, como ao pedir que o usuário digite uma senha correta.

Python
```python
# Exemplo de validação de senha
senha_correta = "python123"
tentativa = input("Digite a senha: ")

while tentativa != senha_correta:
    print("Senha incorreta! Tente novamente.")
    tentativa = input("Digite a senha: ")

print("Acesso concedido!")
```

Cuidado com o Loop Infinito e uso do break

Se a condição do while nunca mudar para falsa, o programa fica preso em repetição contínua (loop infinito). Para interromper a repetição a qualquer momento, usamos o comando break.

Python
```python
# Loop controlado com break
while True:
    comando = input("Digite 'sair' para encerrar o programa: ")
    if comando.lower() == "sair":
        print("Encerrando a aplicação...")
        break
    print(f"Você digitou: {comando}")
```

Mão no Código

Agora é sua vez de exercitar as estruturas de decisão e repetição. Crie os arquivos abaixo no VS Code:

- **Exercício 1 (classificador_notas.py):** Peça a nota de um aluno (0 a 10) e use estruturas condicionais para mostrar a situação: nota >= 9 para "Excelente", entre 7 e 8.9 para "Aprovado", entre 5 e 6.9 para "Recuperação", e abaixo de 5 para "Reprovado".

Exercício 2 (contador_pares.py): Crie um script usando for e range() para percorrer os números de 1 a 20. Mostre apenas os números pares e, ao final, exiba a soma de todos eles.

Capítulo 6: Estruturas de Dados Básicas

Até aqui, cada variável que criamos guardava apenas um valor por vez. No entanto, programas reais precisam trabalhar com grupos de informações: itens em um carrinho de compras, nomes de alunos em uma lista ou dados de um cadastro. Para isso usamos as estruturas de dados.

1. Por que variáveis soltas não bastam?

Se fôssemos guardar as notas de 50 alunos usando apenas variáveis simples, teríamos que criar nota1, nota2, nota3... até nota50. Isso traz problemas claros:
Dificuldade para crescer: Se a turma aumentar para 500 alunos, fica inviável criar centenas de variáveis na mão.

- **Inviabilidade de escala:** E se a turma passasse a ter 500 alunos? Criar centenas de variáveis manualmente tornaria o código ilegível e impossível de manter.
- **Trabalho para calcular:** Para somar e tirar a média, a conta ficaria gigante, citando variável por variável.
- **Quantidade variável:** Em sistemas reais, a quantidade de itens muda o tempo todo, e variáveis soltas não acompanham essa mudança.

As estruturas de dados resolvem isso agrupando diversos valores sob o mesmo nome de variável.

2. Trabalhando com Listas (Lists)

Uma lista em Python junta vários elementos em uma ordem definida. Você pode incluir, alterar e tirar itens quando quiser.

Sintaxe e Índices a partir de Zero
As listas usam colchetes [] e itens separados por vírgula. Cada elemento tem uma posição chamada índice, que começa sempre no zero (0).

Python
```
# Criando uma lista de frutas
frutas = ["Maçã", "Banana", "Laranja", "Uva"]

# Acessando elementos pelo índice
print(frutas[0])  # Exibe "Maçã" (primeiro elemento)
print(frutas[2])  # Exibe "Laranja" (terceiro elemento)
```

Principais Métodos e Funções para Manipulação de Listas
O Python tem várias funções e métodos prontos para usar com listas:

- **len():** Retorna o total de elementos da lista.
- **append():** Adiciona um item no fim da lista.
- **remove():** Exclui um item procurando pelo valor dele.
- **pop():** Remove o item de uma posição específica (ou o último, se não informar a posição).

Python
```
numeros = [10, 20, 30]

# Adicionando um item ao final
numeros.append(40)
print(f"Lista após append: {numeros}")  # [10, 20, 30, 40]

# Verificando o tamanho da lista
print(f"Total de itens: {len(numeros)}")  # 4

# Removendo um item pelo valor
numeros.remove(20)
```

```
print(f"Lista após remove: {numeros}")  # [10, 30, 40]

# Removendo e capturando um item pelo índice
item_removido = numeros.pop(0)
print(f"Item removido: {item_removido}")  # 10
print(f"Lista final: {numeros}")  # [30, 40]
```

Iterando sobre Listas com o Laço for
A forma mais prática de passar por todos os itens de uma lista é usando o laço for:

Python
```
produtos = ["Notebook", "Mouse", "Teclado", "Monitor"]

for produto in produtos:
    print(f"Item disponível em estoque: {produto}")
```

3. Organizando Dados Estruturados com Dicionários

Enquanto as listas organizam itens por números de posição, os dicionários usam pares de chave e valor. Funciona igual a um dicionário de papel: você busca uma palavra (chave) para ler a explicação (valor).

Sintaxe e Manipulação
Os dicionários usam chaves {}, com a chave e o valor separados por dois-pontos (:).

Python
```
# Declarando um dicionário
aluno = {
    "nome": "Beatriz",
    "idade": 24,
    "curso": "Engenharia de Software"
}

# Acessando valores através das chaves
print(f"Nome do aluno: {aluno['nome']}")
```

```
print(f"Curso: {aluno['curso']}")

# Adicionando uma nova chave ou atualizando um valor existente
aluno["media"] = 9.2
aluno["idade"] = 25  # Atualiza o valor de 24 para 25
```

Lista de Dicionários: Modelando Dados do Mundo Real
Juntar listas com dicionários é muito comum. Podemos criar uma lista onde cada item é um dicionário completo com dados de um produto ou de um usuário.

Python
```
# Lista contendo múltiplos dicionários
catalogo_produtos = [
    {"nome": "Smartphone", "preco": 2500.00, "estoque": 12},
    {"nome": "Notebook", "preco": 4500.00, "estoque": 5},
    {"nome": "Fone de Ouvido", "preco": 200.00, "estoque": 30}
]

# Iterando sobre a lista de dicionários
for item in catalogo_produtos:
    print(f"Produto: {item['nome']} | Preço: R$
{item['preco']:.2f}")
```

Mão no Código

Agora teste o uso de listas e dicionários criando o programa abaixo:
Desafio Prático (gerenciador_estoque.py): Crie um controle de estoque simples usando uma lista de dicionários. Cada dicionário deve conter "nome", "quantidade" e "preco_unitario".

- **Desafio Prático (gerenciador_estoque.py):** Crie um programa que simule um sistema de gerenciamento de estoque de uma loja. O programa deve manter uma lista de dicionários, onde cada dicionário representa um produto com as chaves "nome", "quantidade" e "preco_unitario".
    - 1. Cadastre pelo menos 3 produtos iniciais na lista.
    - 2. Use o laço for para percorrer a lista e mostrar os dados de cada item no terminal.

- 3. Calcule o valor total de cada produto (quantidade * preco_unitario) e, no final, mostre o valor acumulado do estoque inteiro.

Capítulo 7: Organizando o Código

À medida que os scripts crescem, colocar tudo em um bloco único dificulta a leitura e a manutenção do código. Neste capítulo, você vai ver como organizar os programas em partes menores e reutilizáveis através das funções.

1. O Problema do Código Repetido

Se você precisa calcular o preço final com imposto de vários produtos em pontos diferentes do programa, escrever a mesma conta toda vez traz desvantagens claras: Essa prática gera três grandes dificuldades:

- **Código poluído:** Repetir trechos parecidos deixa o arquivo longo e difícil de ler.
- **Manutenção demorada:** Se a alíquota do imposto mudar, você terá que alterar a linha em todos os lugares onde a conta foi escrita.
- **Risco de erro:** Quanto mais vezes você digita a mesma lógica, maior a chance de esquecer algum detalhe em um dos pontos.

A solução é aplicar a regra de evitar repetições desnecessárias. Em vez de copiar e colar comandos, isolamos a lógica dentro de uma função nomeada que pode ser chamada sempre que preciso.

2. Criando e Chamando Funções Reutilizáveis

Uma função é um bloco criado para realizar uma tarefa específica. Depois de pronta, ela pode ser usada em qualquer parte do script.

Definindo e Nomeando Funções com def

Criamos funções usando a palavra `def`, o nome da função, parênteses `()` e dois-pontos `:`. Por convenção, usamos nomes em minúsculas com palavras separadas por underline (padrão snake_case).

Python

```
def exibir_boas_vindas():
    print("========================================")
    print("   Bem-vindo ao Sistema de Gestão!      ")
    print("========================================")
```

```
# Chamando a função para ser executada
exibir_boas_vindas()
```

Passando Informações com Parâmetros e Argumentos

Muitas vezes queremos que a função trabalhe com dados diferentes a cada execução. Para isso usamos **parâmetros** dentro dos parênteses da função. Os dados passados no momento de acionar a função são chamados de **argumentos**.

Python
```
# A função recebe o parâmetro 'nome'
def saudar_usuario(nome):
    print(f"Olá, {nome}! É muito bom ter você por aqui.")

# Chamadas passando argumentos diferentes
saudar_usuario("Mariana")
saudar_usuario("Lucas")
```

A Diferença Crucial entre print() e return

Existe uma diferença importante entre exibir uma mensagem na tela e retornar um valor para o programa:

- **Função com print():** Apenas exibe o texto no terminal. Esse valor não fica salvo para uso posterior.
- **Função com return:** Devolve o resultado do cálculo. Esse valor retornado pode ser guardado em uma variável ou usado em outras contas.

Python
```
# Exemplo 1: Função que apenas exibe (não devolve valor)
def calcular_dobro_print(numero):
    print(f"O dobro é: {numero * 2}")

# Exemplo 2: Função que devolve o valor com return
def calcular_dobro_return(numero):
    return numero * 2

# Uso prático do retorno:
resultado = calcular_dobro_return(15)
# Podemos utilizar a variável 'resultado' em novos cálculos
total = resultado + 10
print(f"Resultado final do cálculo: {total}")
```

3. Escopo de Variáveis: Locais vs. Globais

O escopo indica onde uma variável pode ser usada no programa:

- **Escopo Local:** Variáveis criadas dentro de uma função só existem enquanto essa função está rodando. Elas não podem ser acessadas por fora.
- **Escopo Global:** Variáveis criadas fora das funções podem ser lidas em qualquer parte do script.

Veja o exemplo de como uma variável local deixa de existir ao final da função:

```python
def calcular_desconto(preco, percentual):
    # 'desconto' é uma variável LOCAL
    desconto = preco * (percentual / 100)
    preco_final = preco - desconto
    return preco_final

# Chamando a função e guardando o retorno
valor_pago = calcular_desconto(100.0, 15)
print(f"Valor a pagar: R$ {valor_pago:.2f}")

# Tentar acessar a variável 'desconto' diretamente aqui causaria NameError,
# pois ela só existiu temporariamente durante a execução da função.
```

Para enviar uma informação calculada dentro de uma função de volta para o programa principal, use sempre o comando `return`.

Mão no Código

Pratique a criação de funções organizadas montando o script abaixo no VS Code:

- **Desafio Prático (calculadora_combustivel.py):** Crie um programa para calcular os custos de combustível de uma viagem, dividindo a lógica em duas funções:
    - 1. `calcular_litros(distancia, consumo_kml)`: recebe a distância (km) e o consumo (km/l) e retorna o total de litros necessários.
    - 2. `calcular_custo_total(litros, preco_litro)`: recebe os litros e o preço do combustível e retorna o valor final gasto.

- 3. No código principal, peça os dados ao usuário, chame as funções e mostre um resumo formatado com os litros e o custo estimado da viagem.

4. Exiba no terminal os resultados e a confirmação de aprovação ou não da análise.

Capítulo 8: Tratamento de Erros e Boas Práticas

Em algum momento do aprendizado, seu código vai parar de rodar e a tela do terminal vai mostrar mensagens de erro em vermelho. Essa situação é perfeitamente normal e faz parte da rotina de qualquer pessoa que desenvolve software. Entender como ler e corrigir esses avisos é fundamental.

1. Perder o Medo de Errar e Ler o Traceback

Os erros do terminal são indicações diretas do que precisa ser ajustado. Podemos dividir esses avisos em dois tipos principais:

- **Erros de Sintaxe (SyntaxError):** Acontecem quando há algum engano na escrita, como esquecer de fechar parênteses ou faltar dois-pontos em uma instrução. Nesses casos, o Python nem começa a rodar o script.
- **Erros de Execução (Exceções):** A escrita está correta, mas o programa tenta fazer algo impossível enquanto roda, como dividir por zero ou converter letras em números inteiros.

Como ler o relatório de erro (Traceback)
Quando o programa encontra uma falha, o Python mostra um texto chamado Traceback. O ponto principal quase sempre está na última linha, onde ele indica o tipo exato do erro e a linha onde ocorreu.

Python
```
# Exemplo de código que gera uma exceção:
idade = int("vinte")
```

Ao tentar executar a instrução acima, o terminal exibirá uma mensagem semelhante a esta:

None
```
Traceback (most recent call last):
  File "inicio.py", line 2, in <module>
    idade = int("vinte")
```

```
ValueError: invalid literal for int() with base 10: 'vinte'
```

Nesse exemplo, a falha ocorreu na **linha 2** do arquivo `inicio.py` e o motivo foi um **`ValueError`**, pois a palavra "vinte" não pode ser convertida para número direto.

2. Protegendo o Programa com try e except

Para evitar que uma digitação incorreta do usuário faça o programa fechar sozinho, usamos o bloco `try / except`.
Dentro do `try`, colocamos o comando que pode dar erro. Se tudo correr bem, o programa segue normal. Se ocorrer a falha esperada, a execução desvia para o `except`, onde podemos tratar a situação e mostrar um aviso simples.

Python
```
try:
    idade = int(input("Digite sua idade em números: "))
    print(f"Você digitou {idade} anos.")
except ValueError:
    print("Entrada inválida! Por favor, digite apenas números
inteiros (ex: 25).")
```

Sempre indique qual erro você quer tratar (como `ValueError`). Evite exceções gerais sem tipo definido para não esconder outros problemas sem querer.

Combinando try com laços while para validação contínua
Podemos colocar o tratamento de erros dentro de um laço `while` para continuar pedindo o dado até que a informação venha correta:

Python
```
while True:
    try:
        preco = float(input("Digite o preço do produto: R$ "))
        print(f"Preço cadastrado com sucesso: R$ {preco:.2f}")
        break  # Sai do laço se a conversão funcionou
    except ValueError:
        print("Erro: Digite um valor numérico válido, usando
ponto para decimais.")
```

3. Boas Práticas e Padrões de Código Limpo

Além de fazer o programa rodar, é importante escrever um código claro para que outras pessoas consigam entender o que foi feito. No Python, costumamos seguir o guia de estilo PEP 8 para manter o código limpo.

Nomes Significativos e Padrão snake_case

Evite nomes genéricos como `x` ou `n1`. Escolha nomes que expliquem o que a variável guarda. Em Python, use sempre o padrão **snake_case** (letras minúsculas separadas por underline).

Python

```
# Evite nomes confusos e pouco claros:
a = 1500.0
desc = 0.1
v = a - (a * desc)

# Prefira nomes claros e autodidatas:
salario_bruto = 1500.0
taxa_desconto = 0.1
salario_liquido = salario_bruto - (salario_bruto * taxa_desconto)
```

Indentação e Organização Visual

Mantenha 4 espaços por nível de recuo e use linhas em branco para separar partes do código. Isso deixa o arquivo visualmente organizado.

Comentários Inteligentes

Os comentários (usando `#`) servem para explicar o motivo de uma escolha ou de uma regra de negócio, e não para repetir o que o código já deixa evidente.

Python

```
# RUIM: Comentário redundante que apenas repete o que está
escrito
idade = 18  # Atribui o valor 18 à variável idade

# BOM: Comentário que explica o motivo de uma regra de negócio
específica
```

```
# Desconto de 15% aplicado devido à regra fiscal para produtores rurais regionais
aliquota_imposto = 0.85
```

Mão no Código

Aplique a prevenção de erros e as boas práticas no exercício a seguir:

- **Desafio Prático (calculadora_segura.py):** Crie uma calculadora de divisão protegida contra erros de entrada e de conta:
    - 1. Use um laço `while` com `try / except ValueError` para confirmar que os dados digitados são números válidos.
    - 2. Trate a divisão por zero usando `except ZeroDivisionError` com uma mensagem orientativa.

3. Use nomes claros em `snake_case` para todas as variáveis do programa.

Capítulo 9: Projeto Prático Final

Para encerrar os estudos do livro, vamos juntar os conceitos apresentados em um projeto completo: um Gerenciador de Tarefas (To-Do List) rodando direto no terminal.

1. Apresentação do Projeto e Planejamento

Este projeto reúne variáveis, listas, dicionários, funções, estruturas de controle e tratamento de erros no mesmo código.

O programa permite incluir, listar, concluir e remover tarefas. Cada item é representado por um dicionário com o título e o status (concluído ou pendente), e todas as tarefas ficam guardadas em uma lista principal.

2. Construção Passo a Passo

Dividimos as responsabilidades em funções separadas:

- **exibir_menu():** Mostra as opções na tela e recebe a escolha da pessoa.
- **adicionar_tarefa(tarefas):** Pede a descrição da nova tarefa, cria o dicionário e salva na lista.

- **listar_tarefas(tarefas):** Percorre a lista de tarefas e mostra os itens numerados com o status atual.
- **concluir_tarefa(tarefas):** Pede o número da tarefa a ser concluída e atualiza a situação do item.
- **remover_tarefa(tarefas):** Exclui o item escolhido da lista usando o método pop().

A execução do programa fica dentro de um laço while True para manter o menu ativo até a opção de sair ser selecionada.

3. Código-Fonte Completo do Gerenciador de Tarefas

Python

```
def exibir_menu():
    print("\n--- GERENCIADOR DE TAREFAS ---")
    print("1. Adicionar Tarefa")
    print("2. Listar Tarefas")
    print("3. Concluir Tarefa")
    print("4. Remover Tarefa")
    print("5. Sair")
    return input("Escolha uma opção: ")

def adicionar_tarefa(tarefas):
    titulo = input("Digite a descrição da tarefa: ").strip()
    if titulo:
        tarefa = {"titulo": titulo, "concluida": False}
        tarefas.append(tarefa)
        print(f"Tarefa '{titulo}' adicionada com sucesso!")
    else:
        print("A descrição da tarefa não pode estar vazia.")

def listar_tarefas(tarefas):
    if not tarefas:
        print("Nenhuma tarefa cadastrada.")
        return
    print("\n--- LISTA DE TAREFAS ---")
    for indice, tarefa in enumerate(tarefas, start=1):
```

```
        status = "[X]" if tarefa["concluida"] else "[ ]"
        print(f"{indice}. {status} {tarefa['titulo']}")

def concluir_tarefa(tarefas):
    listar_tarefas(tarefas)
    if not tarefas:
        return
    try:
        numero = int(input("Digite o número da tarefa a concluir:
"))
        if 1 <= numero <= len(tarefas):
            tarefas[numero - 1]["concluida"] = True
            print("Tarefa marcada como concluída!")
        else:
            print("Número de tarefa inválido.")
    except ValueError:
        print("Entrada inválida. Digite um número inteiro.")

def remover_tarefa(tarefas):
    listar_tarefas(tarefas)
    if not tarefas:
        return
    try:
        numero = int(input("Digite o número da tarefa a remover:
"))
        if 1 <= numero <= len(tarefas):
            removida = tarefas.pop(numero - 1)
            print(f"Tarefa '{removida['titulo']}' removida com
sucesso!")
        else:
            print("Número de tarefa inválido.")
    except ValueError:
        print("Entrada inválida. Digite um número inteiro.")

def main():
    tarefas = []
```

```
    while True:
        opcao = exibir_menu()
        if opcao == "1":
            adicionar_tarefa(tarefas)
        elif opcao == "2":
            listar_tarefas(tarefas)
        elif opcao == "3":
            concluir_tarefa(tarefas)
        elif opcao == "4":
            remover_tarefa(tarefas)
        elif opcao == "5":
            print("Encerrando o Gerenciador de Tarefas. Até
logo!")
            break
        else:
            print("Opção inválida. Tente novamente.")

if __name__ == "__main__":
    main()
```

Conclusão

Chegar até o fim do livro e construir um projeto completo é um marco importante na sua jornada. Você aprendeu a organizar ideias, estruturar algoritmos e transformar conceitos em código prático.

Aprender a programar é um processo contínuo. Os próximos passos incluem estudar orientação a objetos, explorar bibliotecas da linguagem, trabalhar com arquivos e integrar sistemas a bancos de dados. Mantenha a prática constante criando pequenos projetos no seu dia a dia.